
Hola Mundo

Backend con Java + Spring Boot + PostgreSQL

Tutorial paso a paso

Proyecto	WIN24 – Plataforma de apuestas deportivas
Equipo	Bolirana
Materia	Ingeniería de Software 1 (2016701)
Universidad	Universidad Nacional de Colombia
Stack	Java 21 + Spring Boot 4.0.6 + PostgreSQL 16
ORM	Spring Data JPA (Hibernate)
Fecha	Mayo 8 2026

Integrantes

Samuel Ricardo Pardo Trujillo
Daniel Santiago Rincón Santofimio
Camilo Andrés Salinas Cuervo
Juan Pablo Zuluaga Galindo

Contents

1	¿De que va esto?	2
2	Stack que usamos	2
2.1	¿Por que YAML y no properties?	2
3	Arquitectura en capas	3
3.1	domain/ – las entidades	3
3.2	controllers/ – los endpoints HTTP	3
3.3	services/ – la lógica de negocio	3
3.4	repositories/ – el acceso a datos	4
4	Paso 1: Configurar la base de datos	4
4.1	Crear la BD y el usuario	4
4.2	Datos de conexion	5
5	Paso 2: Crear el proyecto en Spring Initializr	5
6	Paso 3: Configurar application.yml	7
7	Paso 3: Configurar application.yml	7
8	Paso 4: Estructura de carpetas	8
9	Paso 5: Escribir el codigo	9
9.1	domain/Mensaje.java – la entidad	9
9.2	repositories/MensajeRepository.java – el acceso a datos	10
9.3	services/MensajeService.java – la logica de negocio	10
9.4	controllers/MensajeController.java – los endpoints	11
10	Paso 6: Levantar la aplicación	12
11	Paso 7: Probar desde Postman	12
11.1	POST – crear un mensaje	13
11.2	GET – listar mensajes	13
11.3	Verificacion en DBeaver	14
12	Resumen	15

1. ¿De que va esto?

La idea de este tutorial es mostrar como construimos un backend desde cero usando **Java 21 + Spring Boot 3 + PostgreSQL**, siguiendo la arquitectura en capas que definimos como equipo.

Al final del tutorial vamos a tener:

- Una base de datos PostgreSQL conectada a la aplicación.
- Una entidad Java mapeada a una tabla real en la BD.
- Un endpoint POST que recibe datos y los guarda.
- Un endpoint GET que consulta y retorna los datos en JSON.
- La prueba corriendo desde Postman.

2. Stack que usamos

Componente	Tecnologia
Lenguaje	Java 21
Framework	Spring Boot 4.0.6
ORM	Spring Data JPA (Hibernate)
Base de datos	PostgreSQL 16
Build tool	Apache Maven
Cliente HTTP	Postman
IDE de BD	DBeaver

2.1. ¿Por que YAML y no properties?

Spring Boot deja configurar la app con `application.properties` o `application.yml`. Nosotros usamos **YAML** porque es mas legible: en lugar de repetir el prefijo en cada linea, lo agrupa por jerarquia. Cuando el archivo crece con mas configuraciones, la diferencia se nota bastante.

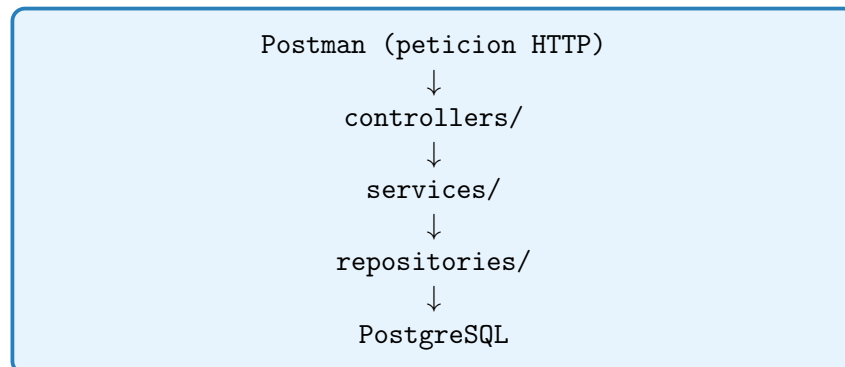
Listing 1: Comparacion: properties vs yaml

```
1 # Con .properties -- repetitivo
2 spring.datasource.url=jdbc:postgresql://localhost:5432/hola_mundo_db
3 spring.datasource.username=jzuluagaga
4 spring.jpa.hibernate.ddl-auto=update
5
6 # Con .yaml -- mas limpio
7 spring:
8   datasource:
9     url: jdbc:postgresql://localhost:5432/hola_mundo_db
10    username: jzuluagaga
11   jpa:
12     hibernate:
```

```
13 ddl-auto: update
```

3. Arquitectura en capas

Antes de escribir código, vale la pena entender como está organizado el proyecto. Usamos una **arquitectura monolítica en capas**, que lo que hace es dividir el código en cuatro responsabilidades claras. Cada capa solo habla con la que tiene debajo, nunca al revés.



3.1. domain/ – las entidades

Lo que hace es representar el mundo real como clases Java. Por ejemplo, si la aplicación maneja mensajes, existe una clase `Mensaje` con sus atributos: `id`, `contenido`, `autor`. Eso es el dominio.

La anotación `@Entity` le dice a JPA: “esta clase es una tabla en la base de datos”. Cada atributo de la clase se convierte en una columna. JPA se encarga de crear esa tabla automáticamente, nosotros solo definimos la estructura.

Lo que no va acá es lógica de negocio. El dominio no sabe si un mensaje es válido, no sabe si el usuario tiene permisos, no sabe nada de eso. Solo representa los datos, nada más. Es como el plano de una casa: describe cómo es, no decide quién puede entrar.

3.2. controllers/ – los endpoints HTTP

Lo que hace es ser la puerta de entrada de la aplicación. Cuando alguien hace una petición desde Postman, un navegador o el frontend, lo primero que recibe es el controller.

El controller no procesa nada, no valida nada, no toca la base de datos. Lo único que hace es tres cosas: recibir la petición, pasársela al service para que haga el trabajo, y devolverle la respuesta al cliente.

Es como el mesero del restaurante: anota el pedido, se lo lleva a la cocina y trae lo que la cocina preparó. El mesero no cocina. Por eso la regla es clara: si en un controller hay lógica de negocio, algo está mal. Eso le pertenece al service.

3.3. services/ – la lógica de negocio

Lo que hace es ser el cerebro de la aplicación. Toda la lógica vive acá: las validaciones, las reglas del negocio, los cálculos. Si hay que verificar que un dato sea correcto antes

de guardarlo, eso pasa en el service. Si hay que combinar información de dos fuentes distintas, eso también pasa en el service.

El service llama al repository cuando necesita leer o guardar datos, pero el no sabe nada de HTTP. No sabe si la petición vino de Postman, de un navegador o de una app móvil. No le importa. El solo recibe datos, hace su trabajo y retorna el resultado.

Es como la cocina del restaurante: prepara los platos sin saber quien los pidio ni como llegaron al local.

3.4. repositories/ – el acceso a datos

Lo que hace es ser el único punto de contacto con la base de datos. Si alguien necesita guardar algo, consultar algo o borrar algo de la BD, tiene que pasar por aca.

La magia esta en que al extender `JpaRepository`, Spring nos genera automaticamente los metodos mas comunes: `save()` para guardar, `findAll()` para traer todo, `findById()` para buscar por id, `deleteById()` para borrar. Todo eso sin escribir una sola linea de SQL.

Si necesitamos algo mas especifico, lo declaramos aca tambien. Pero la regla es clara: nadie mas toca la BD directamente. Ni el service, ni el controller. Solo el repository. Es como el almacen de un restaurante: solo el encargado del almacen sabe donde esta cada cosa.

Ojo

La regla que no se puede romper: las dependencias solo van hacia abajo. Un controller llama a un service. Un service llama a un repository. Nunca al reves, nunca saltandose una capa.

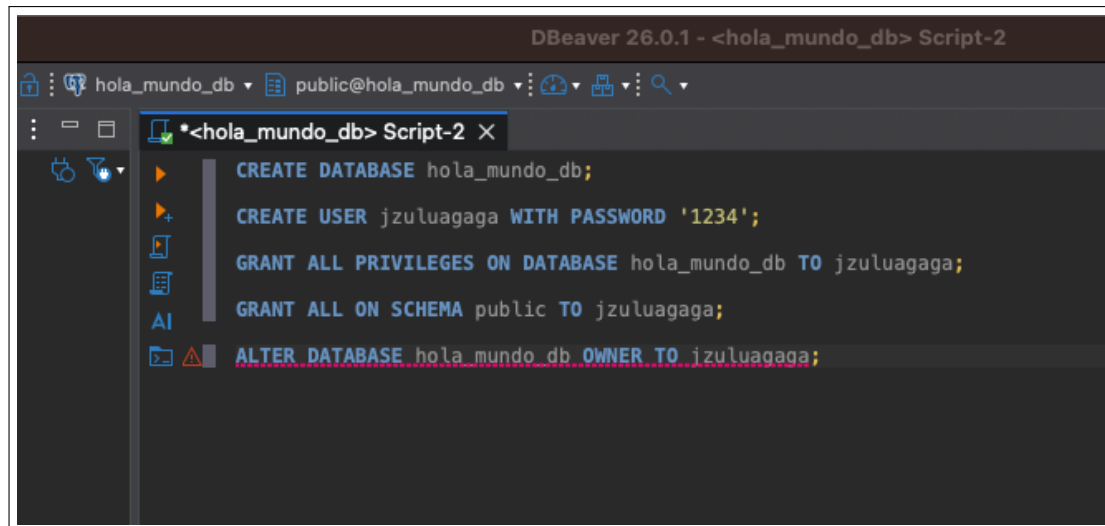
4. Paso 1: Configurar la base de datos

4.1. Crear la BD y el usuario

Abrimos DBeaver conectados con el usuario `postgres` (el superusuario) y ejecutamos:

Listing 2: Crear BD y usuario en PostgreSQL

```
1  -- Creamos la base de datos
2  CREATE DATABASE hola_mundo_db;
3
4  -- Creamos un usuario dedicado para la aplicacion
5  CREATE USER jzuluagaga WITH PASSWORD '1234';
6
7  -- Le damos permisos sobre la BD
8  GRANT ALL PRIVILEGES ON DATABASE hola_mundo_db TO jzuluagaga;
9
10 -- En PostgreSQL 16 tambien hay que dar permisos sobre el schema
11 GRANT ALL ON SCHEMA public TO jzuluagaga;
12
13 -- Lo hacemos dueno de la BD
14 ALTER DATABASE hola_mundo_db OWNER TO jzuluagaga;
```



Buena practica

Nunca conectar la aplicacion directamente con el usuario **postgres**. Lo que hacemos es crear un usuario dedicado con solo los permisos que necesita. En produccion la contrasena tiene que ser segura, no 1234.

4.2. Datos de conexion

Con esto ya tenemos todo lo que necesitamos para conectar Spring Boot:

Parametro	Valor
Host	localhost
Puerto	5432
Base de datos	hola_mundo_db
Usuario	jzuluagaga
Contraseña	1234 (solo para desarrollo)

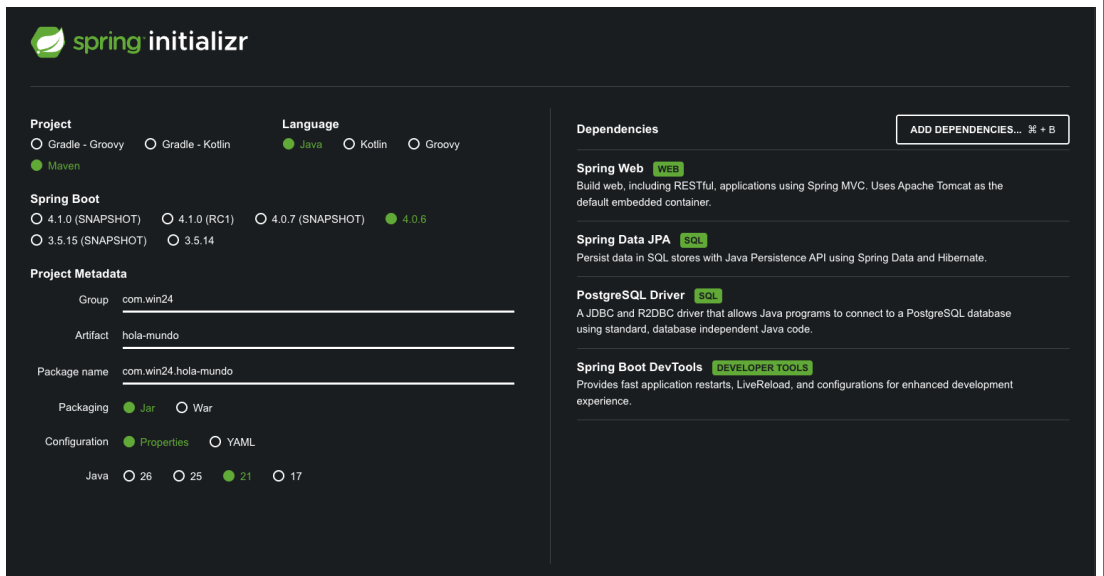
5. Paso 2: Crear el proyecto en Spring Initializr

Vamos a <https://start.spring.io> y configuramos el proyecto asi:

Campo	Valor
Project	Maven
Language	Java
Spring Boot	4.0.6
Group	com.win24
Artifact	hola-mundo
Packaging	Jar
Java	21

Las dependencias que agregamos son:

- **Spring Web** – lo que hace es incluir el servidor HTTP embebido (Tomcat). Sin esto no hay endpoints.
- **Spring Data JPA** – lo que hace es conectar Java con la BD a traves de Hibernate como ORM.
- **PostgreSQL Driver** – el conector JDBC para hablar con PostgreSQL específicamente.
- **Spring Boot DevTools** – lo que hace es recargar la app automaticamente cuando guardamos cambios. Util en desarrollo.



The screenshot shows the Spring Initializr web application interface. It is divided into two main sections: Project Metadata and Dependencies.

Project Metadata:

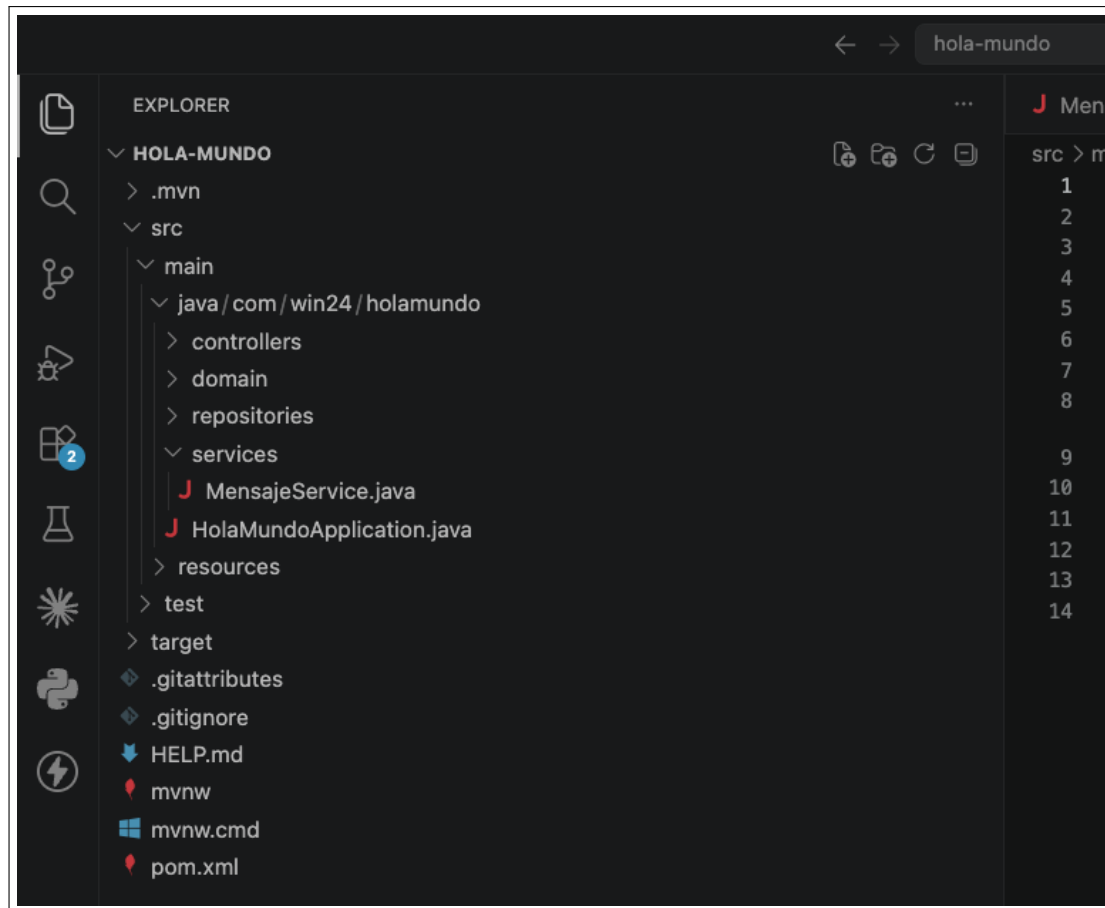
- Project:** ☐ Gradle - Groovy, ☐ Gradle - Kotlin, ☒ Java, ☐ Kotlin, ☐ Groovy
- Language:** ☒ Maven
- Spring Boot:** ☐ 4.1.0 (SNAPSHOT), ☐ 4.1.0 (RC1), ☐ 4.0.7 (SNAPSHOT), ☒ 4.0.6
- Project Metadata:**
 - Group:
 - Artifact:
 - Package name:
 - Packaging: ☒ Jar, ☐ War
 - Configuration: ☒ Properties, ☐ YAML
 - Java: ☐ 26, ☐ 25, ☒ 21, ☐ 17

Dependencies:

- Spring Web** (WEB): Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.
- Spring Data JPA** (SQL): Persist data in SQL stores with Java Persistence API using Spring Data and Hibernate.
- PostgreSQL Driver** (SQL): A JDBC and R2DBC driver that allows Java programs to connect to a PostgreSQL database using standard, database independent Java code.
- Spring Boot DevTools** (DEVELOPER TOOLS): Provides fast application restarts, LiveReload, and configurations for enhanced development experience.

There is an "ADD DEPENDENCIES..." button in the top right corner of the Dependencies section.

Descargamos el ZIP, lo descomprimos y lo abrimos en el Visual Studio Code.



6. Paso 3: Configurar application.yml

7. Paso 3: Configurar application.yml

Eliminamos el archivo `application.properties` que viene por defecto y creamos `src/main/resources/application.yml` con el siguiente contenido:

Listing 3: `src/main/resources/application.yml`

```

1 # --- Servidor -----
2 server:
3   port: 8080
4
5 # --- Base de datos PostgreSQL -----
6 spring:
7   datasource:
8     url: jdbc:postgresql://localhost:5432/hola_mundo_db
9     username: jzuluagaga
10    password: "1234"
11    driver-class-name: org.postgresql.Driver
12
13 # --- JPA / Hibernate -----
14 jpa:
15   hibernate:

```



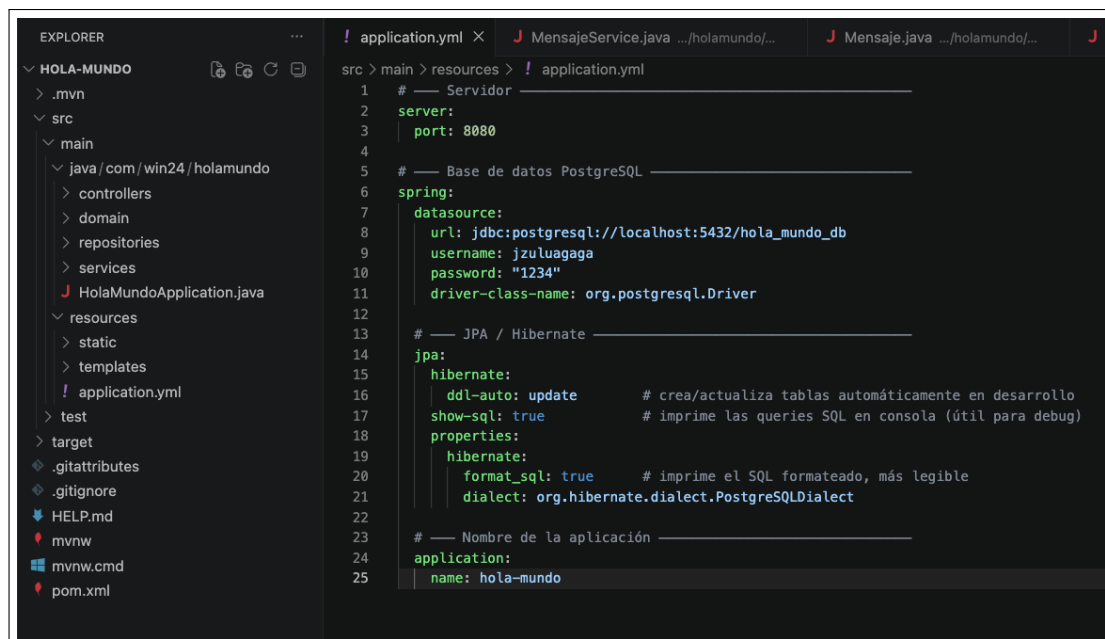
```

16     ddl-auto: update      # crea/actualiza tablas automaticamente
17     show-sql: true       # muestra las queries en consola
18     open-in-view: false  # evita el warning de open-in-view
19     properties:
20       hibernate:
21         format_sql: true
22         dialect: org.hibernate.dialect.PostgreSQLDialect
23
24     # --- Nombre de la app -----
25     application:
26       name: hola-mundo

```

Ojo

El `ddl-auto: update` lo que hace es crear las tablas automaticamente al arrancar la app basandose en las entidades `@Entity` que encuentre. Esto solo se usa en desarrollo. En produccion se cambia a `validate` y se manejan migraciones con Flyway.



8. Paso 4: Estructura de carpetas

Dentro del paquete base del proyecto creamos cuatro paquetes nuevos. La estructura queda así:

Listing 4: Estructura del backend

```

1  src/main/java/com/win24/holamundo/
2    controllers/
3      MensajeController.java
4    services/
5      MensajeService.java
6    repositories/
7      MensajeRepository.java
8    domain/

```

```
9      Mensaje.java
10     HolaMundoApplication.java    <- ya existe, no tocar
```

9. Paso 5: Escribir el código

9.1. domain/Mensaje.java – la entidad

Lo que hace es representar un mensaje en la aplicación y decirle a JPA que cree una tabla mensajes en PostgreSQL con las columnas id, contenido y autor.

Listing 5: domain/Mensaje.java

```
1 package com.win24.holamundo.domain;
2
3 import jakarta.persistence.*;
4
5 @Entity
6 @Table(name = "mensajes")
7 public class Mensaje {
8
9     @Id
10    @GeneratedValue(strategy = GenerationType.IDENTITY)
11    private Long id;
12
13    @Column(nullable = false)
14    private String contenido;
15
16    @Column(nullable = false)
17    private String autor;
18
19    // Constructor vacío -- requerido por JPA
20    public Mensaje() {}
21
22    // Constructor con los campos
23    public Mensaje(String contenido, String autor) {
24        this.contenido = contenido;
25        this.autor = autor;
26    }
27
28    // Getters y setters
29    public Long getId() { return id; }
30    public void setId(Long id) { this.id = id; }
31
32    public String getContenido() { return contenido; }
33    public void setContenido(String contenido) {
34        this.contenido = contenido;
35    }
36
37    public String getAutor() { return autor; }
38    public void setAutor(String autor) { this.autor = autor; }
39 }
```

9.2. repositories/MensajeRepository.java – el acceso a datos

Lo que hace es darnos los metodos para hablar con la BD sin escribir SQL. Al extender JpaRepository ya tenemos gratis save(), findAll(), findById() y mas.

Listing 6: repositories/MensajeRepository.java

```
1 package com.win24.holamundo.repositories;
2
3 import com.win24.holamundo.domain.Mensaje;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface MensajeRepository extends JpaRepository<Mensaje,
9     Long> {
10     // No necesitamos escribir nada mas por ahora.
11     // JpaRepository ya nos da save(), findAll(), findById(), etc.
12 }
```

9.3. services/MensajeService.java – la logica de negocio

Lo que hace es coordinar las operaciones. Aca vivira toda la logica del sistema. Por ahora solo crea y lista mensajes, pero en el proyecto real aca iran las validaciones y reglas de negocio.

Listing 7: services/MensajeService.java

```
1 package com.win24.holamundo.services;
2
3 import com.win24.holamundo.domain.Mensaje;
4 import com.win24.holamundo.repositories.MensajeRepository;
5 import org.springframework.stereotype.Service;
6 import java.util.List;
7
8 @Service
9 public class MensajeService {
10
11     private final MensajeRepository mensajeRepository;
12
13     // Inyeccion por constructor -- buena practica,
14     // no usar @Autowired directamente en el campo
15     public MensajeService(MensajeRepository mensajeRepository) {
16         this.mensajeRepository = mensajeRepository;
17     }
18
19     public Mensaje crear(String contenido, String autor) {
20         Mensaje mensaje = new Mensaje(contenido, autor);
21         return mensajeRepository.save(mensaje);
22     }
23
24     public List<Mensaje> listarTodos() {
25         return mensajeRepository.findAll();
26     }
27 }
```

9.4. controllers/MensajeController.java – los endpoints

Lo que hace es exponer dos rutas HTTP al mundo exterior. Recibe la petición, llama al service y retorna la respuesta. Nada mas.

Listing 8: controllers/MensajeController.java

```
1 package com.win24.holamundo.controllers;
2
3 import com.win24.holamundo.domain.Mensaje;
4 import com.win24.holamundo.services.MensajeService;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8 import java.util.List;
9
10 @RestController
11 @RequestMapping("/api/mensajes")
12 public class MensajeController {
13
14     private final MensajeService mensajeService;
15
16     public MensajeController(MensajeService mensajeService) {
17         this.mensajeService = mensajeService;
18     }
19
20     // POST /api/mensajes?contenido=...&autor=...
21     @PostMapping
22     public ResponseEntity<Mensaje> crear(
23         @RequestParam String contenido,
24         @RequestParam String autor) {
25         Mensaje nuevo = mensajeService.crear(contenido, autor);
26         return ResponseEntity.status(HttpStatus.CREATED).body(nuevo);
27     }
28
29     // GET /api/mensajes
30     @GetMapping
31     public ResponseEntity<List<Mensaje>> listar() {
32         return ResponseEntity.ok(mensajeService.listarTodos());
33     }
34 }
```

Ojo

En este tutorial retornamos la entidad **Mensaje** directamente desde el controller para mantenerlo simple. Pero en el proyecto real esto no se hace así, y vale la pena entender por que.

Imagina que la entidad **Usuario** tiene un campo **password**. Si retornamos la entidad directo por HTTP, ese campo va a aparecer en la respuesta JSON. Eso es un problema de seguridad enorme. Además, puede que la BD tenga columnas internas que el cliente no necesita ver para nada.

Lo que hacemos en el proyecto real es usar DTOs: objetos dedicados a la transferencia de datos. **MensajeRequest** es lo que recibimos del cliente, y **MensajeResponse** es exactamente lo que le queremos mostrar, ni mas ni menos. Es como un filtro: la entidad es tuya, el DTO es lo que decides mostrarle al mundo.

10. Paso 6: Levantar la aplicación

Con todo el código listo, levantamos el servidor desde la terminal dentro de la carpeta del proyecto:

Listing 9: Comando para levantar el backend

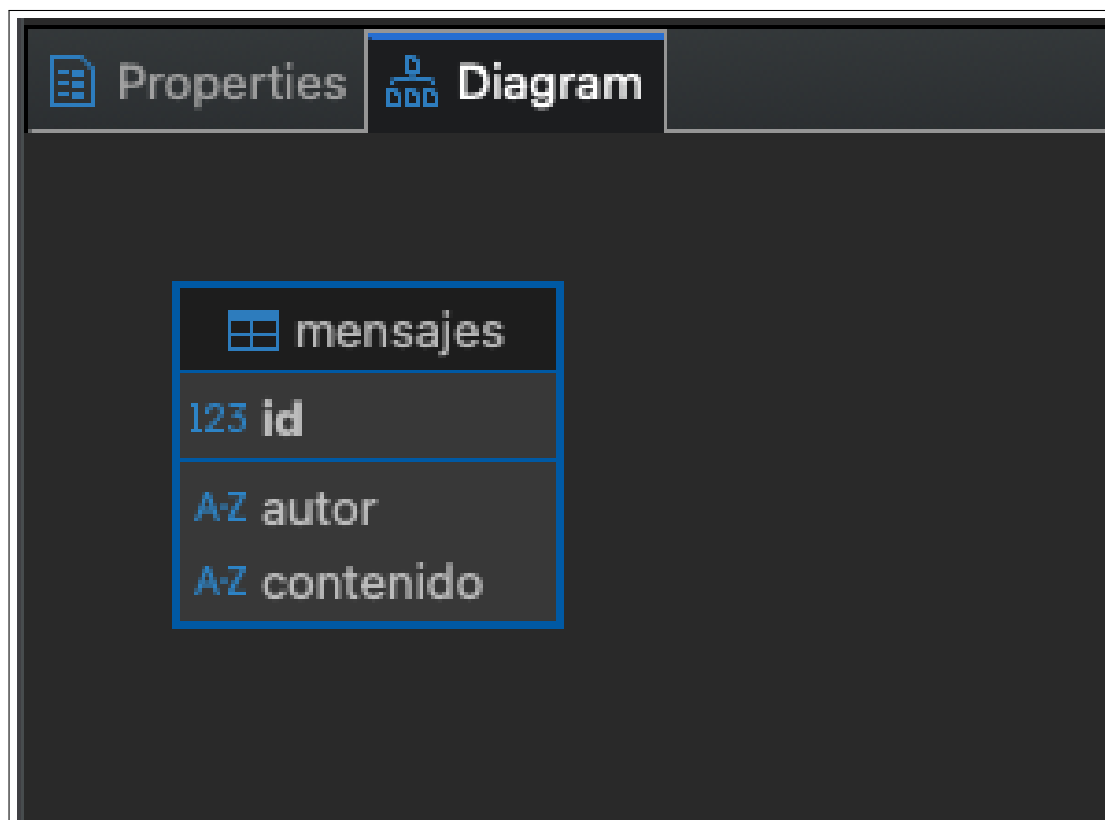
```
1 mvn spring-boot:run
```

Si todo esta bien, en la consola vamos a ver algo así:

Listing 10: Salida esperada en consola

```
1 Hibernate: create table mensajes (...)  
2 Tomcat started on port 8080 (http) with context path '/'  
3 Started HolaMundoApplication in 1.8 seconds
```

La línea de Hibernate confirmando `create table mensajes` nos dice que JPA creo la tabla automáticamente en PostgreSQL. La de Tomcat nos dice que el servidor esta listo y escuchando en `http://localhost:8080`.



La tabla se crea automáticamente según lo definido en domain.

11. Paso 7: Probar desde Postman

11.1. POST – crear un mensaje

Abrimos Postman y configuramos la petición así:

Campo	Valor
Metodo	POST
URL	http://localhost:8080/api/mensajes
Params	contenido = Hola Mundo autor = Juan Pablo

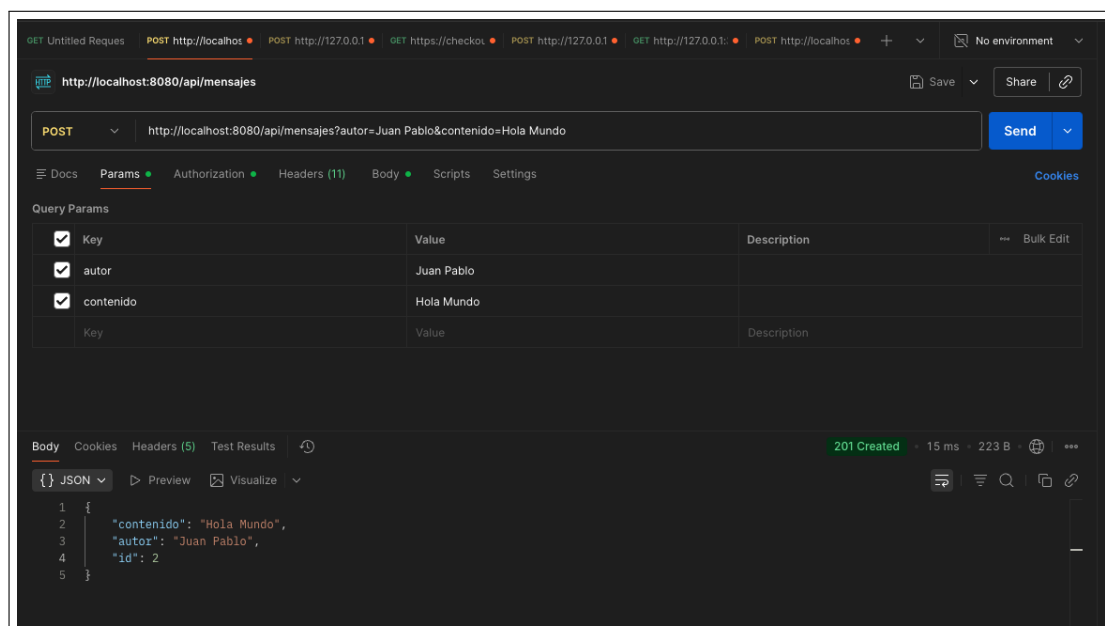
Lo que hace `@RequestParam` es leer los parametros directamente desde la URL, por eso van en la pestaña **Params** y no en el Body. La URL queda así:

```
1 http://localhost:8080/api/mensajes?contenido=Hola Mundo&autor=Juan Pablo
```

Al darle Send, el servidor responde con 201 Created y el objeto guardado, incluyendo el id generado automaticamente por PostgreSQL:

Listing 11: Respuesta exitosa del POST

```
{
  "id": 1,
  "contenido": "Hola Mundo",
  "autor": "Juan Pablo"
}
```



11.2. GET – listar mensajes

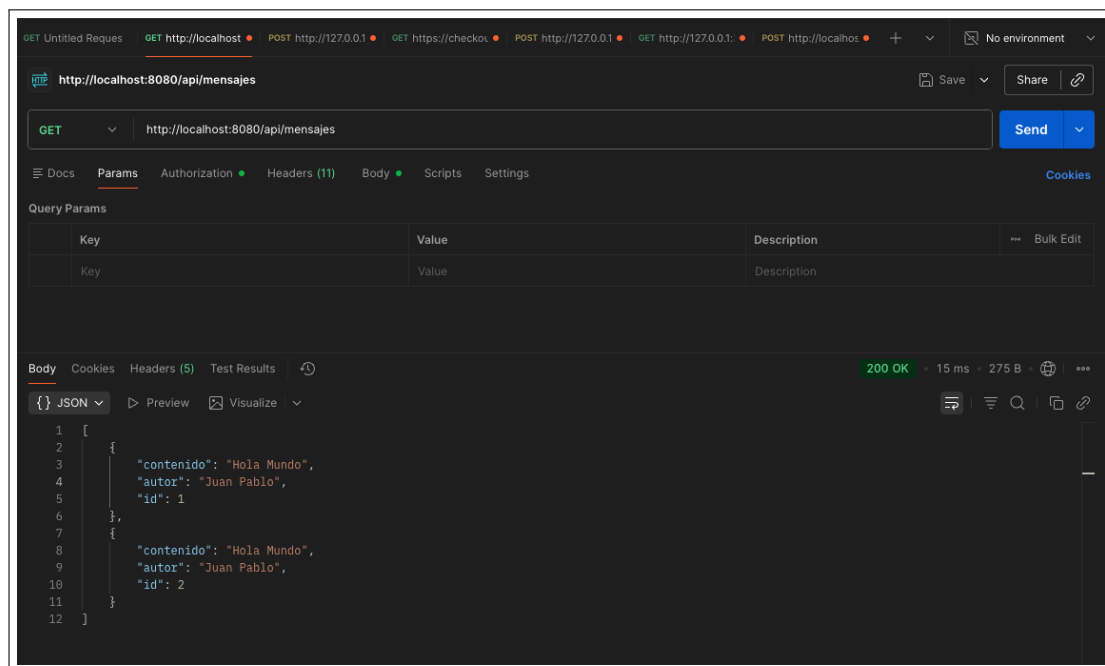
Ahora verificamos que el dato realmente se guardo en la BD haciendo un GET:

Campo	Valor
Metodo	GET
URL	http://localhost:8080/api/mensajes

Sin params, sin body. El servidor responde con 200 OK y un arreglo JSON con todos los mensajes guardados:

Listing 12: Respuesta exitosa del GET

```
[
  {
    "id": 1,
    "contenido": "Hola Mundo",
    "autor": "Juan Pablo"
  }
]
```

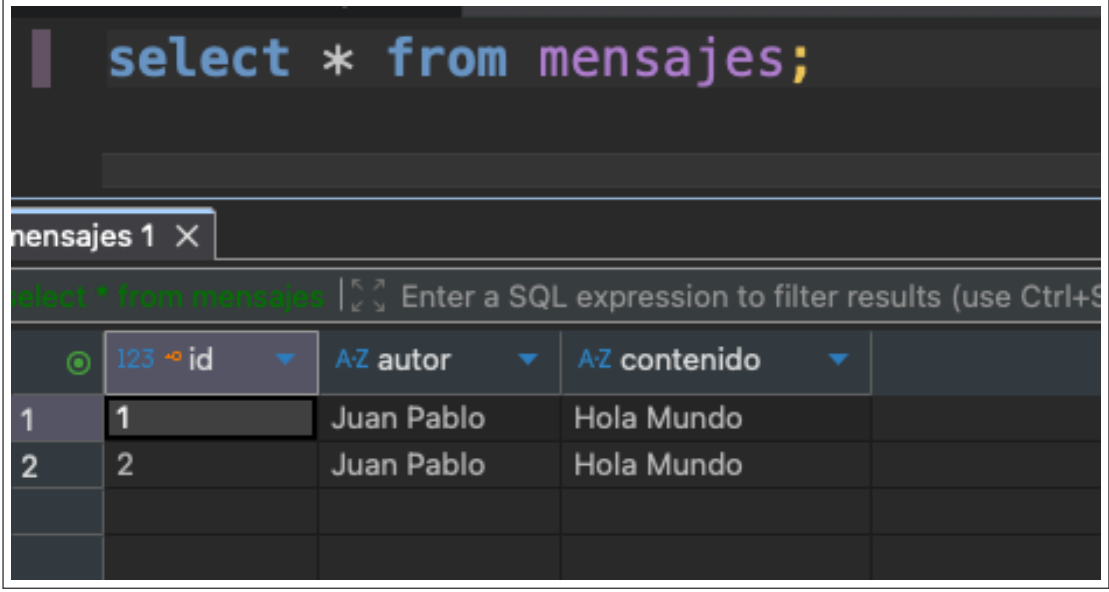


11.3. Verificación en DBeaver

Para cerrar el ciclo, podemos confirmar en DBeaver que el registro existe en la base de datos real:

Listing 13: Consulta directa a PostgreSQL

```
1 SELECT * FROM mensajes;
```



The screenshot shows a SQL query editor with the query `select * from mensajes;` entered. Below the query, there is a table titled 'mensajes 1' with the following data:

	123 id	AZ autor	AZ contenido
1	1	Juan Pablo	Hola Mundo
2	2	Juan Pablo	Hola Mundo

12. Resumen

Con esto tenemos un backend funcional que:

1. Se conecta a PostgreSQL y crea las tablas automaticamente.
2. Recibe una petición HTTP POST, guarda el dato en la BD y responde con el objeto creado.
3. Recibe una petición HTTP GET, consulta la BD y retorna los datos en JSON.

El flujo completo de una petición POST es:

```

Postman envia POST /api/mensajes?contenido=Hola&autor=Juan
↓
MensajeController recibe la petición y llama a MensajeService.crear()
↓
MensajeService crea el objeto Mensaje y llama a MensajeRepository.save()
↓
MensajeRepository ejecuta INSERT en PostgreSQL
↓
PostgreSQL guarda la fila y retorna el id generado
↓
Postman recibe 201 Created con {"id":1,"contenido":"Hola
Mundo","autor":"Juan"}

```

Buena practica

Este Hola Mundo es la base de todo el proyecto WIN24. La arquitectura en capas que usamos aca (controllers, services, repositories, domain) es exactamente la misma que vamos a escalar para Usuarios, Apuestas, Eventos y Pagos. La unica diferencia es que en el proyecto real usamos DTOs en lugar de retornar las entidades directamente.